

- There are multiple level user to database
 - ⌘ **User ---- DNS --- Web --- Application --- DB**
 - ⌘ At user's browser level, caching happens. Means browser store the DNS information so that next time it'll not have to request for resolving the domain.
 - ⌘ At **Web** layer, **CDN** remains. Lets assume you want to serve some static files keeps in S3 bucket.
 - ⌘ So, first when you get the file, store it in a place and serve those files when required without going to the S3 bucket.
 - ⌘ This is done with **CDN**. (Content Delivery Network)
 - ⌘ At **Application** layer, you can use **Redis**, **Memcached** or some other caching tools so that you'll not have to make DB calls multiple times to fetch the same data.
 - ⌘ Even at **DB** layer you can do caching.
- **DNS**
 - ⌘ When a user requests a website, their system or browser queries DNS servers to translate the domain name into an IP address.
 - ⌘ Caching these results means that repeated requests for the same domain are resolved faster, reducing the need for repetitive network looks up and improving overall browsing efficiency.
 - ⌘ This is particularly beneficial for reducing latency and enhancing user experience by avoiding delays in domain resolution.
- **Web**
 - ⌘ Accelerate retrieval of web content from web/app servers.
 - ⌘ Web caching enhances performance by storing copies of frequently accessed web pages, images, and other content.
 - ⌘ When users request content, the cache delivers it quickly without needing to retrieve it from the origin server, which reduces load times and server strain.
 - ⌘ This not only speeds up content delivery but also reduces bandwidth usage and server costs.
 - ⌘ This is typically implemented using CDN.
 - ⌘ Web caching is crucial for improving the responsiveness of web applications and ensuring a smoother, faster user experience.
 - ⌘ **What it caches? : Static files (images, CSS, JS), HTML pages, Public API responses**
- **Application** caching (Redis, Memcached, ..etc)

- ⌘ What it caches? : DB query results, User sessions, Computed objects, Aggregations, Authorization info
- Simple difference between **CDN** and **Redis/Memcached/** (other application service) is:
 - ⌘ **CDN** caches the things between **client** and **server**
 - ⌘ **Redis / Memcached** cache data **inside your backend**, before hitting the database.

Browser ⇌ CDN ⇌ App Server ⇌ Redis ⇌ Database

- ⌘ **Cache terminologies**
 - ⌘ Cache HIT and Cache MISS
 - ⌘ Cache eviction policies: LRU, LFU, FIFO, TTL bases
 - ⌘ Cache expiry and TTL
 - ⌘ Cache loading strategies: Lazy loading, Eager loading or Cache warming
 - ⌘ Cache size and capacity
- F

Caching in Spring Boot

➤ There are the below annotations.

♣ @EnableCaching

- ♣ It's a **class** level annotation.
- ♣ It should be written in a component.
- ♣ Avoid writing this in the main method's application class, otherwise it'll make **caching mandatory**.
- ♣ By default it setup a **CacheManager** and creates *in-memory* cache using one *concurrent Hashmap*.

♣ @Cacheable

- ♣ It's a **method** level annotation.
- ♣ Its used with a method whose response has to be cached.
- ♣ Spring boot manages the request and response of the method to the cache that is specified in the annotation attribute.
- ♣ **It just see the cache, if the data present then just return that.**

♣ @CachePut

- ♣ It is also a **method** level annotation.
- ♣ It is used to update the cache **after invoking the method**.
- ♣ By doing this, the result is put in the cache and the method is executed.
- ♣ It has same attributes of **@Cacheable** annotation.

♣ @CacheEvict

- ♣ It is also a **method** level annotation.
- ♣ It is used to **remove the data from the cache**.
- ♣ When the method is annotated with this annotation, then the method is executed and the cache will be removed/evicted.

➤ Example

```
@Component
public class MyMathService {

    @Cacheable("piDecimals")
    public int computePiDecimal(int precision) {
        ...
    }
}
```

- Cache is just like a **HashMap** which stores in *key-value* pair.

- ⌘ Normally redis stores data like this

```
users::10 → {"id":10,"name":"Alok","email":"alok@example.com"}
```

- ⌘ Lets assume the method is

```
@Cacheable(value = "users", key = "#id")
public User getUser(Long id) {
    return new User(10, "Alok", "alok@example.com");
}
```

- ⌘ Here, the “value” field inside @Cacheable (**not the value inside redis**) represents the method (getUser) and “key” field inside @Cacheable represents the argument.
- ⌘ These both combined makes the **key of Redis**, and the **value of Redis** against this key will be like

```
{"id":10,"name":"Alok","email":"alok@example.com"}
```

- ⌘ So, “**value**” field inside @Cacheable is not the *value* of Redis cache.
- ⌘ Instead of value, you can write cacheNames as well. Both are aliases of each other.

- F

➤ Implementing Redis caching in spring boot application

- First create a config file and annotate it with `@EnableCaching`, because you shouldn't write this annotation in the main method's class.

```
@Configuration
@EnableCaching
public class CacheConfig {
}
```

- Annotate the cacheable method with `@Cacheable`

```
@Cacheable(value = "employee", key = "#id")
public EmployeeDto getEmployeeById(Long id) {
```

- ⚡ Note that #, without this it'll not take the value of the argument.
- ⚡ Here, Redis key will be like: **employee::3** (if id = 3)
- ⚡ If you want to be more specific, like if the method has more arguments and you want to be specific with more arguments then

```
@Cacheable(value = "employee", key = "{#id, #name}")
public EmployeeDto getEmployeeById(Long id, String name) {
```

it'll create keys like: **employee::[5, alok]** (id = 5, name = "alok")

- ⚡ Its SePL query, **{ }** means list.
- ⚡ It'll become a **composite key**. (id, name) (Object composite)
- ⚡ **It is safer to use.**
- ⚡ Also, another method of writing this is

```
@Cacheable(value = "employee", key = "#id ':' #name}")
public EmployeeDto getEmployeeById(Long id, String name) {
```

it'll create keys like: **employee:5:alok**

- ⚡ Its also **composite key** (String composite)
- ⚡ If you have a lot of filters (to be passed inside *key* field) then you can use a custom key generator function and pass like

```
@Cacheable(value = "employee", keyGenerator = "customKeyGen")
public EmployeeDto getEmployeeById(Long id, String name) {
```

- ⚡ It sends all the arguments to that **customKeyGen** method.
- ⚡ One more method is *hashing*, you can create a token and store that as the **key in redis cache**.
- ⚡ Either you can use custom key generator method where you'll return the hash.

- Or you can use SePL language directly

```
@Cacheable(value = "search", key = "T(com.example.HashUtil).sha256(#request.toString())")
```

- Annotate with **@CachePut** annotation

```
@CachePut(cacheNames = "employee", key = "#result.id")  
public EmployeeDto createNewEmployee( @NotNull EmployeeDto employeeDto)
```

```
@CachePut(cacheNames = "employee", key = "#id")  
public EmployeeDto updateEmployee(Long id, @NotNull EmployeeDto employeeDto)
```

- ⌘ **#result.id** because the return type of *createNewEmployee* method is *EmployeeDto*,

```
@Data 24 usages  Alok Ranjan Joshi  
public class EmployeeDto implements Serializable {  
    private Long id;  
    private String email;  
    private String name;  
    private Long salary;  
}
```

even if it is private and the getter method is *getId()*, so instead of **#result.id** we should write **#result.getId()**, but NO.

Internally SePL invokes the getter method like **#result.getId()** when we write **#result.id**

- ⌘ **Now, whenever we'll create a new employee or update an employee, it'll write the latest data into the cache after the method is executed.**
- ⌘ **NOTE: it'll store the data in Cache only if the method is executed successfully, in case of any exception it doesn't store the data in cache.**

- Annotate with **@CacheEvict** annotation

```
@CacheEvict(cacheNames = "employee", key = "#id")  
public void deleteEmployee(Long id) {
```

- ⌘ It'll delete the data from the cache if the method *deleteEmployee* is executed successfully.



Internal working of Caching in Spring Boot

- There is an interface **CacheManager** which has to be implemented by the cache provider, in our case it is **Redis**

```
public interface CacheManager { 3 usages
    @Nullable 6 implementations
    Cache getCache(String name);

    Collection<String> getCacheNames();
}
```

```
public class RedisCacheManager extends AbstractTransactionSupportingCacheManager {
    protected static final boolean DEFAULT_ALLOW_RUNTIME_CACHE_CREATION = true;
    private final boolean allowRuntimeCacheCreation;
    private final RedisCacheConfiguration defaultCacheConfiguration;
    private final RedisCacheWriter cacheWriter;
    private final Map<String, RedisCacheConfiguration> initialCacheConfiguration;
```

(**AbstractTransactionSupportingCacheManager** extends **CacheManager**)

- **defaultCacheConfiguration** : it is to set different types of configs like prefix, expiry, reset the expiry if the cache hits again ..etc

```
public class RedisCacheConfiguration { no usages
    protected static final boolean DEFAULT_CACHE_NULL_VALUES = true;
    protected static final boolean DEFAULT_ENABLE_TIME_TO_IDLE_EXPIRATION = false;
    protected static final boolean DEFAULT_USE_PREFIX = true;
    protected static final boolean DO_NOT_CACHE_NULL_VALUES = false;
    protected static final boolean DO_NOT_USE_PREFIX = false;
    protected static final boolean USE_TIME_TO_IDLE_EXPIRATION = true;
    private final boolean cacheNullValues;
    private final boolean enableTimeToIdle;
    private final boolean usePrefix;
    private final CacheKeyPrefix keyPrefix;
    private final ConversionService conversionService;
    private final RedisSerializationContext.SerializationPair<String> keySerializationPair;
    private final RedisSerializationContext.SerializationPair<Object> valueSerializationPair;
    private final RedisCacheWriter.TtlFunction ttlFunction;
```

- **initialCacheConfiguration** : it is to set the default configurations for different caches, like for “employee” one type of configuration, for “department” another type of configuration

```
Map<String, RedisCacheConfiguration> configs = new HashMap<>()

configs.put("employee",
    RedisCacheConfiguration.defaultCacheConfig()
        .entryTtl(Duration.ofMinutes(10)));

configs.put("users",
    RedisCacheConfiguration.defaultCacheConfig()
        .entryTtl(Duration.ofHours(1)));
```

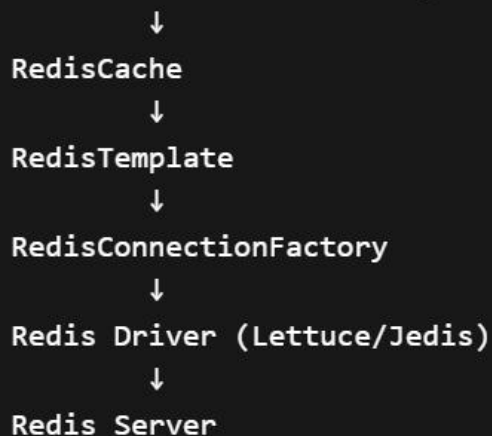

- This **CacheManager**'s bean is auto-configured by spring, if you want custom configuration then you can create the bean of **RedisCacheManager**.
- This is how you can create custom **RedisCacheManager** bean

```
@Bean new *
public CacheManager cacheManager(RedisConnectionFactory redisConnectionFactory) {
    RedisCacheConfiguration redisCacheConfiguration = RedisCacheConfiguration.defaultCacheConfig()
        .prefixCacheNameWith( prefix: "my-test-")
        .entryTtl( ttl: Duration.ofSeconds( seconds: 60))
        .enableTimeToIdle()
        .serializeKeysWith(
            keySerializationPair: RedisSerializationContext
                .SerializationPair
                .fromSerializer( serializer: new StringRedisSerializer())
        )
        .serializeValuesWith(
            valueSerializationPair: RedisSerializationContext
                .SerializationPair
                .fromSerializer( serializer: new GenericJackson2JsonRedisSerializer())
        );

    return RedisCacheManager.builder(redisConnectionFactory)
        .cacheDefaults( defaultCacheConfiguration: redisCacheConfiguration)
        .build();
}
```

- ⋈ **RedisConnectionFactory** (implemented by **LettuceConnectionFactory**) creates and manages connections that Spring uses to talk to Redis.
 - ⋈ Without this spring cannot execute GET, SET, DEL, TTL, ..etc.
- ⋈ **RedisCacheConfiguration** is the one which are (mentioned in the previous page), it adds the default configurations for the redis cache manager.
- ⋈ **serialize** are for the type in which keys and values will be stored in redis db.

Your Code / CacheInterceptor



-
-

- Internal Flow
- CacheAutoConfiguration (auto-configuration class)
 - ⌘ CacheManagerCustomizers, CacheManagerValidator,
 - ⌘ CacheManagerCustomizers contains list of CacheManagerCustomizer
 - ⌘ CacheManagerCustomizer takes generic (T extends CacheManager), it contains a manager *customize* which takes CacheManager or one of its sub classes as its argument.
 - ⌘ So, if we create CacheManager bean by our own, then it'll take that CacheManager bean instead of the default one.
 - ⌘ Now the question is, we create only one bean of CacheManager, then how it's a list

```
public class CacheManagerCustomizers { no usages
    private final List<CacheManagerCustomizer<?>> customizers;
```

- ⌘ So, this is not list of CacheManager itself, its a list of CacheManagerCustomizer

```
@FunctionalInterface no usages
public interface CacheManagerCustomizer<T extends CacheManager> {
    void customize(T cacheManager);
}
```

- ⌘ We can have multiple CacheManagerCustomizer

```
@Bean
CacheManagerCustomizer<RedisCacheManager> ttlCustomizer() {
    return cm -> cm.setDefaultExpiration(Duration.ofMinutes(5));
}

@Bean
CacheManagerCustomizer<RedisCacheManager> metricsCustomizer() {
    return cm -> enableMetrics(cm);
}
```

- F

- Spring Boot is built on this philosophy:

“The framework owns object creation,
developers only tweak behavior.”

Spring scans beans



Finds CacheManagerCustomizer bean



CacheAutoConfiguration collects it via ObjectProvider



Adds it to CacheManagerCustomizers list



When CacheManager is created



Boot calls customizer.customize(cacheManager)

-

Customizer

- Customizer interface is to customize a already created object with some different parameters. Consider the below example

```
public class Server {  
  
    private int port = 8080;  
    private int timeout = 30;  
    private boolean compressionEnabled = false;  
    private String environment = "dev";  
    private boolean metricsEnabled = false;  
  
    // setters  
    public void setPort(int port) { this.port = port; }  
    public void setTimeout(int timeout) { this.timeout = timeout; }  
    public void setCompressionEnabled(boolean compressionEnabled) {  
        this.compressionEnabled = compressionEnabled;  
    }  
    public void setEnvironment(String environment) { this.environment = environment; }  
    public void setMetricsEnabled(boolean metricsEnabled) { this.metricsEnabled = metricsEnabled; }  
  
    @Override  
    public String toString() {  
        return "Server{" +  
            "port=" + port +  
            ", timeout=" + timeout +  
            ", compressionEnabled=" + compressionEnabled +  
            ", environment='" + environment + '\'' +  
            ", metricsEnabled=" + metricsEnabled +  
            '}';  
    }  
}
```

```
@FunctionalInterface  
public interface ServerCustomizer {  
    void customize(Server server);  
}
```

```
public class PortCustomizer implements ServerCustomizer {  
    @Override  
    public void customize(Server server) {  
        server.setPort(9090);  
    }  
}
```

```
public class TimeoutCustomizer implements ServerCustomizer {
    @Override
    public void customize(Server server) {
        server.setTimeout(60);
    }
}
```

```
public class CompressionCustomizer implements ServerCustomizer {
    @Override
    public void customize(Server server) {
        server.setCompressionEnabled(true);
    }
}
```

```
public class MetricsCustomizer implements ServerCustomizer {
    @Override
    public void customize(Server server) {
        server.setMetricsEnabled(true);
    }
}
```

```
public class EnvironmentCustomizer implements ServerCustomizer {
    @Override
    public void customize(Server server) {
        server.setEnvironment("production");
    }
}
```

- Customizer takes the actual object that has to be customized, and customize its params so that you need not change the properties of the object manually.

```
import java.util.List;

public class ServerFactory {

    public static Server createServer(List<ServerCustomizer> customizers) {

        // Framework creates object with defaults
        Server server = new Server();

        // Apply all customizers
        for (ServerCustomizer customizer : customizers) {
            customizer.customize(server);
        }

        return server;
    }
}
```

- Server factory takes list of customizers (here `ServerCustomizer`; because it is the customizer interface of the `Server` class) and create an object and change its values according to those customizers.
- You can have multiple customizers for multiple properties of an object.

```
public static void main(String[] args) {

    List<ServerCustomizer> customizers = List.of(
        new PortCustomizer(),
        new TimeoutCustomizer(),
        new CompressionCustomizer(),
        new EnvironmentCustomizer(),
        new MetricsCustomizer()
    );

    Server server = ServerFactory.createServer(customizers);

    System.out.println(server);
}
```

- Now, you'll take all the customizers (that you need) in a list and create the object and call the *customize* method of all those customizers to configure your created object.

➤ HENCE; Customizer doesn't help in creation of an object (bean in spring boot). rather they changes the properties of that object after the object being created.

- In spring security, when we have to configure the *formLogin* part, we can directly pass `Customizer.withDefaults()` which by default set the properties

```
http.formLogin(Customizer.withDefaults());
```

instead we had to write like the below

```
http.formLogin(form -> {});
```

```
@Bean
SecurityFilterChain security(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth.anyRequest().authenticated())
        .formLogin(Customizer.withDefaults())
        .httpBasic(Customizer.withDefaults());

    return http.build();
}
```

- ⌘ This **Customizer.withDefaults()** also returns a Customizer object.

```
public interface Customizer<T> {  
  
    void customize(T t);  
  
    static <T> Customizer<T> withDefaults() {  
        return t -> { };    // no changes  
    }  
}
```

- ⌘ formLogin() takes customizer and customize the created object according to that.

➤ F

➤ **CacheAutoConfiguration** class is the auto configuration class of Caching in Spring Boot.

- ♣ It creates bean of **CacheManagerCustomizers**.
- ♣ CacheManagerCustomizers class contains list of CacheManagerCustomizer objects
- ♣ But mostly you don't need to create beans of this **CacheManagerCustomizer**. Because customizers are only to update something in the pre-created beans (objects).
- ♣ If you want to create your own custom cache manager (which is done mostly), you can create a bean of **CacheManager**.

```
@Bean new *
public CacheManager cacheManager(RedisConnectionFactory redisConnectionFactory) {
    RedisCacheConfiguration redisCacheConfiguration = RedisCacheConfiguration.defaultCacheConfig()
        .prefixCacheNameWith( prefix: "my-test-")
        .entryTtl( ttl: Duration.ofSeconds( seconds: 60))
        .enableTimeToIdle()
        .serializeKeysWith(
            keySerializationPair: RedisSerializationContext
                .SerializationPair
                .fromSerializer( serializer: new StringRedisSerializer())
        )
        .serializeValuesWith(
            valueSerializationPair: RedisSerializationContext
                .SerializationPair
                .fromSerializer( serializer: new GenericJackson2JsonRedisSerializer())
        );

    return RedisCacheManager.builder(redisConnectionFactory)
        .cacheDefaults( defaultCacheConfiguration: redisCacheConfiguration)
        .build();
}
```

- ♣ There is a class called **CacheConfigurations** which contains a mapper of cache type to the specific cache provider

```
static {
    Map<CacheType, String> mappings = new EnumMap( keyType: CacheType.class);
    mappings.put(CacheType.GENERIC, GenericCacheConfiguration.class.getName());
    mappings.put(CacheType.HAZELCAST, HazelcastCacheConfiguration.class.getName());
    mappings.put(CacheType.INFINISPAN, InfinispanCacheConfiguration.class.getName());
    mappings.put(CacheType.JCACHE, JCacheCacheConfiguration.class.getName());
    mappings.put(CacheType.COUCBASE, CouchbaseCacheConfiguration.class.getName());
    mappings.put(CacheType.REDIS, RedisCacheConfiguration.class.getName());
    mappings.put(CacheType.CAFFEINE, CaffeineCacheConfiguration.class.getName());
    mappings.put(CacheType.CACHE2K, Cache2kCacheConfiguration.class.getName());
    mappings.put(CacheType.SIMPLE, SimpleCacheConfiguration.class.getName());
    mappings.put(CacheType.NONE, NoOpCacheConfiguration.class.getName());
    MAPPINGS = Collections.unmodifiableMap( m: mappings);
}
```


- These classes are the configuration classes which creates the Cache bean.

```
@Configuration(  
    proxyBeanMethods = false  
)  
@ConditionalOnBean({Cache.class})  
@ConditionalOnMissingBean({CacheManager.class})  
@Conditional({CacheCondition.class})  
class GenericCacheConfiguration {  
    @Bean  
    SimpleCacheManager cacheManager(CacheManagerCustomizers customizers, Collection<Cache> caches) {  
        SimpleCacheManager cacheManager = new SimpleCacheManager();  
        cacheManager.setCaches(caches);  
        return (SimpleCacheManager) customizers.customize(cacheManager);  
    }  
}
```

➤ F

ACID Properties

- ACID properties description
 - ⌘ **Atomicity**
 - ⌘ Transaction is **all-or-nothing**. Either all the operations succeed, or none do.
 - ⌘ **Consistency**
 - ⌘ Transaction brings DB from one **valid state to another**.
 - ⌘ Example:
 - * A had 100, B had 50; after transferring 50 to B, A will have 50 and B will have 100.
 - * Total money before: 150, Total money after: 150
 - ⌘ **Isolation**
 - ⌘ Transaction do not interfere with each other.
 - ⌘ Example: flight booking system, 2 people shouldn't end up booking the same seat.
 - ⌘ **Durability**
 - ⌘ If transaction is committed, even after DB or System failure, the changes should be permanent.
- Advantages
 - ⌘ Data Integrity: data remains consistent and free from corruption.
 - ⌘ Reliability: consistent and reliable execution of transactions.
 - ⌘ Concurrency control: simultaneous access to data without conflicts.
 - ⌘ Fault tolerance: data durability, surviving system failures.
 - ⌘ Transaction management: structured transaction handling.
- Disadvantages
 - ⌘ Performance overhead: impact system performance & throughput due to additional processing and resource utilization
 - ⌘ Complexity: add complexity to DB systems, increasing design and maintenance challenges.
 - ⌘ Scalability challenges
 - ⌘ Potential for deadlocks
 - ⌘ Limited concurrency
-

➤ Example of Transaction in PostgreSQL

- ⌘ You can open the postgresql terminal using **psql** command in your terminal.

```
root@48cf3c202770:/# psql --username lovableUser -d lovabledb
```

- ⌘ NOTE: you need to enter **;** otherwise it'll wait till you enter this.
- ⌘ I created a dummy table *accounts* and entered a few values

```
lovabledb=# create table accounts ( name varchar(100), balance decimal(10, 2) );
CREATE TABLE
lovabledb=# INSERT INTO accounts (name, balance)
VALUES
('Rahul', 500.00),
('Neha', 1200.75),
('Amit', 300.25);
INSERT 0 3
```

- ⌘ To start a transaction: **begin;** is used

```
lovabledb=# begin;
BEGIN
lovabledb=#
```

- ⌘ If you see in the below, when I did **rollback;** the state got reverted.

Whatever you write after **begin;** its just a snapshot that gets updated, not the DB.

```
lovabledb=# begin;
BEGIN
lovabledb=# select * from accounts;
 name | balance
-----+-----
Rahul | 500.00
Neha  | 1200.75
Amit  | 300.25
(3 rows)

lovabledb=# update accounts set balance=400.00 where name='Rahul';
UPDATE 1
lovabledb=# select * from accounts;
 name | balance
-----+-----
Neha  | 1200.75
Amit  | 300.25
Rahul | 400.00
(3 rows)

lovabledb=# rollback;
ROLLBACK
lovabledb=# select * from accounts;
 name | balance
-----+-----
Rahul | 500.00
Neha  | 1200.75
Amit  | 300.25
(3 rows)
```

name='rahul' or name='rahul' , double quotes for identifiers, single quote for strings.

- ♣ In the below, I have done **commit;** it'll update the table.

```
lovabledb=# begin;
BEGIN
lovabledb=# select * from accounts;
name | balance
-----+-----
Rahul | 500.00
Neha  | 1200.75
Amit  | 300.25
(3 rows)

lovabledb=# update accounts set balance = 400.00 where name = 'Rahul';
UPDATE 1
lovabledb=# select * from accounts;
name | balance
-----+-----
Neha  | 1200.75
Amit  | 300.25
Rahul | 400.00
(3 rows)

lovabledb=# commit;
COMMIT
lovabledb=# select * from accounts;
name | balance
-----+-----
Neha  | 1200.75
Amit  | 300.25
Rahul | 400.00
(3 rows)
```

- ♣ In the below, you can see we got one error while executing the sql query. So even if you execute **commit;** here it'll rollback to its original state.

```
lovabledb=# begin;
BEGIN
lovabledb=# select * from accounts;
name | balance
-----+-----
Neha  | 1200.75
Amit  | 300.25
Rahul | 400.00
(3 rows)

lovabledb=# update accounts set balance=200 where name = 'Amit';
UPDATE 1
lovabledb=# select * from accounts;
name | balance
-----+-----
Neha  | 1200.75
Rahul | 400.00
Amit  | 200.00
(3 rows)

lovabledb=# select * from accountsssss;
ERROR:  relation "accountsssss" does not exist
LINE 1: select * from accountsssss;
                        ^
lovabledb=# commit;
ROLLBACK
lovabledb=# select * from accounts;
name | balance
-----+-----
Neha  | 1200.75
Amit  | 300.25
Rahul | 400.00
(3 rows)
```

Transactional Isolation

- When multiple transactions run at same time, DB must decide: “Who sees whose changes and when”

- Isolation Levels:

♣ READ UNCOMMITTED

- ✎ It allows a transaction to see the changes made by other transactions even if they are NOT committed yet.

- ✎ You can read the data that might still be rolled back.

- ✎ **Account Balance = 1000** (initial state)

```
Transaction T1
-----
BEGIN
UPDATE account SET balance = 500  ← not committed yet

Transaction T2
-----
SELECT balance → 500  !  (dirty read)

T1 → ROLLBACK
```

- ✎ Here T1 will rollback if he wants or he gets any error; reading data by T2 will not cause any error in T1.

- ✎ In PostgreSQL, a transaction never fails because another transaction reads its data — failures only occur due to errors **within** the transaction or concurrency conflicts like deadlocks or serialization failures.

- ✎ PostgreSQL treats **READ UNCOMMITTED** as **READ COMMITTED**.

♣ READ COMMITTED (default in PostgreSQL)

- ✎ A transaction NEVER sees uncommitted versions created by other transactions.

- ✎ PostgreSQL uses MVCC (Multi-Version Concurrency Control) to implement READ UNCOMMITTED.

* The DB doesn't overwrite the old row.

* Rather, it creates a new version of that row.

*

id	balance	version
1	100	v1 (old)
1	150	v2 (new)

 (like this)

- * You can't see those versions directly; PostgreSQL manages those internally using metadata (xmin, xmax)

- Initially the balance was: 100

```
## MVCC Example – PostgreSQL (Read Committed)

### Initial state
balance = 100
```

- Then transaction T1 update the balance 150 but not yet committed

```
### Step 1 – Transaction T1 updates

T1:
BEGIN
UPDATE balance = 150  (not committed)
```

- Now PostgreSQL will keep 2 versions of this row

```
**Internal row versions:**

v1 → balance 100 (committed)
v2 → balance 150 (uncommitted)
```

- Now, another transaction T2 wants to read that data

```
### Step 2 – Transaction T2 reads

T2 runs:

SELECT balance FROM accounts WHERE id = 1;
```

- The rule is: a transaction can get the version of data which is of latest version and committed.

Now, latest version is v1 (in our case) because version v2 is not yet committed.

So, T2 will read the balance as 100 (not 150)

```
PostgreSQL snapshot logic:

- v2 is uncommitted → invisible
- Latest committed version is v1

✅ Result returned to T2 → **100**
```

- If T1 commits its changes, version v2 becomes the committed version. So when T2 reads the data again, it will now see 150 instead of 100.

- ⌘ In READ COMMITTED, every query inside a transaction fetches the latest committed data at that moment. So if a transaction runs multiple queries, each one can return *different* values if other transactions *commit* changes in between.

⌘ REPEATABLE READ

- ⌘ Repeatable Read removes the inconsistency you can face in READ COMMITTED, where the same row might return different values if other transactions commit changes in between.
- ⌘ It works by taking a snapshot at the **start** of the transaction. Even if other transactions update and commit new values later, the current transaction continues to see the data exactly as it existed *when it began*.
- ⌘ Initially the balance was 100

```
### Initial state
balance = 100
```

- ⌘ Transaction T1 is started, it'll get the value: 100

```
### Step 1 - T1 starts (Repeatable Read)

```sql
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT balance; -- returns 100
```

- ⌘ Now another transaction T2 comes, update the balance, and commits it.

```
BEGIN;
UPDATE accounts SET balance = 200;
COMMIT;
```

- ⌘ Now, if T1 reads the balance again, it'll get 100 only (not 200)

```
Step 3 - T1 reads again

```sql
SELECT balance FROM accounts;
-- Output: 100
```

- ⌘ If it was READ COMMITTED, then here T1 would have gotten 200 instead of 100.

⌘ SERIALIZABLE

⌘



- Above all the things were for **READ**, how the **WRITE** happens?
 - ⌘ **READ UNCOMMITTED**
 - ⌘ Here, if some transaction updates a row, it writes normally (with LOCKS)
 - ⌘ But the database does a very little protection;
 - ⌘ Other transaction can **OVERWRITE** the same row without strong checks
 - ⌘ Two transaction can update based on old values;
 - ⌘ Writes don't coordinate much, so overwriting can happen.
 - ⌘ **READ COMMITTED**
 - ⌘ When a transaction updates a row, it apply LOCK.
 - ⌘ If another transaction tries to update the same row, it must wait until the first one commits.
 - ⌘ After waiting, it'll update the latest committed value.
 - ⌘ No dirty writes; updates happens one after another; but if your logic reads faster than write then overwrite still may happen.
 - ⌘ **REPEATABLE READ**
 - ⌘ In many databases, Read Committed and Repeatable Read use **similar locks for writes**, but when conflicts happen, Read Committed typically **waits and proceeds**, while Repeatable Read may either **block or abort** the transaction depending on the database
 - ⌘ Some DBs have same write rule for both **READ COMMITTED** and **REPEATABLE READ**.
- In simple words:
 - ⌘ **READ UNCOMMITTED**
 - ⌘ You can read even uncommitted data; writes may overwrite — least safe.
 - ⌘ **READ COMMITTED**
 - ⌘ You read only committed data; writes wait and update the latest value.
 - ⌘ **REPEATABLE READ**
 - ⌘ You read a fixed snapshot; writes allowed only if data hasn't changed (else wait/abort).
 - ⌘ **SERIALIZABLE**
 - ⌘ Transactions behave as if run one by one; conflicts cause retries — safest.
 - ⌘ Example:
 - * Account balance is 100.
 - * There are 2 transactions A and B. A increase the money by 15 and B decrease by 25.

- * If the sequence is A -- B => 100 ---A--- 115 ---B--- 90
- * If the sequence is B -- A => 100 ---B--- 75 ---A--- 90
- * So, both are same, but what if A and B are running simultaneously
100 ---A--- 115
---B--- 75
- * It'll give error, so Postgres (where isolation level is **SERIALIZABLE**
READ) abort the 2nd transaction's update.

⤵

➤ F

- In PostgreSQL, READ UNCOMMITTED doesn't exist, even if you set the transaction isolation level as READ UNCOMMITTED it'll treat that like READ COMMITTED.
- To see the current transaction isolation

```
lovabledb=# show transaction isolation level;
transaction_isolation
-----
read committed
(1 row)
```

by default postgres has **read committed** as the transaction isolation level.

- I started 2 transactions

T1

```
lovabledb=# begin;
BEGIN
lovabledb=#
```

T2

```
lovabledb=# begin;
BEGIN
lovabledb=#
```

- In both transactions if I query a table, it'll give the latest committed state

```
lovabledb=# select * from accounts;
 name | balance
-----+-----
Neha  | 1200.75
Amit   | 300.25
Rahul  | 400.00
(3 rows)
```

(in both T1 & T2)

- Now, lets update Amit's balance to 350 in **T1**.

```
lovabledb=# update accounts set balance=350.00 where name='Amit';
UPDATE 1
lovabledb=# select * from accounts;
 name | balance
-----+-----
Neha  | 1200.75
Rahul  | 400.00
Amit   | 350.00
(3 rows)
```

If we query the table from T2 it'll display the previous value only (300.25) as isolation state is READ COMMITTED (as T1 has not committed yet so T2 will not be able to see the changes done by T1)

```
lovabledb=# select * from accounts;
 name | balance
-----+-----
Neha  | 1200.75
Amit   | 300.25
Rahul  | 400.00
(3 rows)
```

(T2)

- Now if T1 commits its changes, then in T2 also we can see the latest values.

T1

```
lovabledb=# commit;
COMMIT
```

T2

```
lovabledb=# select * from accounts;
 name | balance 
-----+-----
 Neha | 1200.75
 Rahul |  400.00
 Amit  |  350.00
(3 rows)
```

(now its 350.00)

- -----
- Now lets again do the same thing, till T1 updates

T1

```
lovabledb=# begin;
BEGIN
lovabledb=# select * from accounts;
 name | balance 
-----+-----
 Neha | 1200.75
 Rahul |  400.00
 Amit  |  350.00
(3 rows)

lovabledb=# update accounts set balance=800.00 where name='Amit';
UPDATE 1
lovabledb=# select * from accounts;
 name | balance 
-----+-----
 Neha | 1200.75
 Rahul |  400.00
 Amit  |  800.00
(3 rows)
```

T2

```
lovabledb=# begin;
BEGIN
lovabledb=# select * from accounts;
 name | balance 
-----+-----
 Neha | 1200.75
 Rahul |  400.00
 Amit  |  350.00
(3 rows)
```

T1 updated the value 800.00 but not committed yet; so T2 can see 350.00 only

- Now if T2 updates the same value again, it'll not be possible because T1 has already updated the table;

So, T2 will wait till T1 commits its latest changes.

T2

```
lovabledb=# update accounts set balance=650.00 where name='Amit';
|
```

It'll not update the value until T1 commits.

- Now if T1 commit its changes then T2's waiting state will end and it'll update the table.

```
T1 lovabledb=*# commit;  
COMMIT
```

```
T2 lovabledb=*# update accounts set balance=650.00 where name='Amit';  
UPDATE 1
```

- Still, we'll see the value as 800.00 only as T1 updated it and committed. Until T2 commits it, the db will not be updated.

```
lovabledb=# select * from accounts;  
name | balance  
-----+-----  
Neha  | 1200.75  
Rahul | 400.00  
Amit  | 800.00  
(3 rows)
```

(actual db; not inside any

transaction)

- Now if T2 commits its changes, then the DB will be updated.

```
T2 lovabledb=*# commit;  
COMMIT
```

```
lovabledb=# select * from accounts;  
name | balance  
-----+-----  
Neha  | 1200.75  
Rahul | 400.00  
Amit  | 650.00  
(3 rows)
```

(actual DB state)

===== REPEATABLE READ =====

- Start 2 transactions T1 and T2, and set the transaction isolation level

REPEATABLE READ

```
lovabledb=# begin;
BEGIN
lovabledb=# show transaction isolation;
transaction_isolation
-----
read committed
(1 row)

lovabledb=# set transaction isolation level repeatable read;
SET
lovabledb=# show transaction isolation;
transaction_isolation
-----
repeatable read
(1 row)
```

T1 & T2

- In T1, update the table

```
lovabledb=# select * from accounts;
 name | balance
-----+-----
Neha  | 1200.75
Rahul | 400.00
Amit  | 650.00
(3 rows)

lovabledb=# update accounts set balance=250.00 where name='Amit';
UPDATE 1
lovabledb=# select * from accounts;
 name | balance
-----+-----
Neha  | 1200.75
Rahul | 400.00
Amit  | 250.00
(3 rows)
```

Now in T2, we can see the previous old value only

```
lovabledb=# select * from accounts;
 name | balance
-----+-----
Neha  | 1200.75
Rahul | 400.00
Amit  | 650.00
(3 rows)
```

- Now even if T1 commits, T2 will still see the old value only

```
lovabledb=# commit;
COMMIT
lovabledb=# select * from accounts;
 name | balance
-----+-----
Neha  | 1200.75
Rahul | 400.00
Amit  | 250.00
(3 rows)
```


In T2, if we get the data, we'll still see the old value (because REPEATABLE KEY takes the value from the latest committed snapshot)

```
lovabledb=# select * from accounts;
 name | balance 
-----+-----
 Neha | 1200.75
 Rahul |  400.00
 Amit  |  650.00
(3 rows)
```

- Now, if T2 tries to update the value, it'll abort because it contains a stale value (in case of COMMITTED READ it allows the update because T1 has already committed)

```
lovabledb=# update accounts set balance=100.00 where name='Amit';
ERROR:  could not serialize access due to concurrent update
lovabledb=# |
```

(T2)

- Now even if we commit T2, it'll ROLLBACK because one error has occurred.

```
lovabledb=# commit;
ROLLBACK
```

- But, if you are updating different rows in both the transactions, then it'll not abort.

```
lovabledb=# begin; 1
BEGIN
lovabledb=# show transaction isolation level; 2
transaction_isolation
-----
read committed
(1 row)

lovabledb=# set transaction isolation level repeatable read; 3
SET
lovabledb=# show transaction isolation level; 4
transaction_isolation
-----
repeatable read
(1 row)

lovabledb=# select * from accounts; 5
 name | balance 
-----+-----
 Neha | 1200.75
 Amit  |  330.00
 Rahul |  360.00
(3 rows)

lovabledb=# update accounts set balance=balance+35 where name='Amit'; 6
UPDATE 1
lovabledb=# select * from accounts; 8
 name | balance 
-----+-----
 Neha | 1200.75
 Rahul |  360.00
 Amit  |  365.00 ←
(3 rows)

lovabledb=# commit; 10
COMMIT
lovabledb=# select * from accounts; 12
 name | balance 
-----+-----
 Neha | 1200.75
 Rahul |  360.00
 Amit  |  365.00 ←
(3 rows)

lovabledb=#
```

```
lovabledb=# begin; 1
BEGIN
lovabledb=# show transaction isolation level; 2
transaction_isolation
-----
read committed
(1 row)

lovabledb=# set transaction isolation level repeatable read; 3
SET
lovabledb=# show transaction isolation level; 4
transaction_isolation
-----
repeatable read
(1 row)

lovabledb=# select * from accounts; 5
 name | balance 
-----+-----
 Neha | 1200.75
 Amit  |  330.00
 Rahul |  360.00
(3 rows)

lovabledb=# update accounts set balance=balance+25 where name='Rahul'; 7
UPDATE 1
lovabledb=# select * from accounts; 9
 name | balance 
-----+-----
 Neha | 1200.75
 Amit  |  330.00
 Rahul |  385.00 ←
(3 rows)

lovabledb=# commit; 11
COMMIT
lovabledb=# select * from accounts; 13
 name | balance 
-----+-----
 Neha | 1200.75
 Amit  |  365.00
 Rahul |  385.00 ←
(3 rows)

lovabledb=#
```

- F

===== SERIALIZABLE =====

```

lovabledb=# begin;
BEGIN
lovabledb=# show transaction isolation level;
transaction_isolation
-----
read committed
(1 row)

lovabledb=# set transaction isolation level serializable;
SET
lovabledb=# show transaction isolation level;
transaction_isolation
-----
serializable
(1 row)

lovabledb=# select * from doctor;
 name | on_call
-----+-----
  Neha | t
   Amit | t
  Rahul | t
(3 rows)

lovabledb=# select * from doctor where on_call=true;
 name | on_call
-----+-----
  Neha | t
   Amit | t
  Rahul | t
(3 rows)

lovabledb=# update doctor set on_call=false where name='Neha';
UPDATE 1
lovabledb=# select * from doctor where on_call=true;
 name | on_call
-----+-----
   Amit | t
  Rahul | t
(2 rows)

lovabledb=# commit;
COMMIT
lovabledb=# select * from doctor;
 name | on_call
-----+-----
   Amit | t
  Rahul | t
  Neha | f
(3 rows)
lovabledb=#

```

only one ✓

```

lovabledb=# begin;
BEGIN
lovabledb=# show transaction isolation level;
transaction_isolation
-----
read committed
(1 row)

lovabledb=# set transaction isolation level serializable;
SET
lovabledb=# show transaction isolation level;
transaction_isolation
-----
serializable
(1 row)

lovabledb=# select * from doctor;
 name | on_call
-----+-----
  Neha | t
   Amit | t
  Rahul | t
(3 rows)

lovabledb=# select * from doctor where on_call=true;
 name | on_call
-----+-----
  Neha | t
   Amit | t
  Rahul | t
(3 rows)

lovabledb=# update doctor set on_call=false where name='Rahul';
UPDATE 1
lovabledb=# select * from doctor where on_call=true;
 name | on_call
-----+-----
  Neha | t
   Amit | t
(2 rows)

lovabledb=# commit;
ERROR:  could not serialize access due to read/write dependencies among transactions
DETAIL:  Reason code: Canceled on identification as a pivot, during commit attempt.
HINT:  The transaction might succeed if retried.
lovabledb=#
lovabledb=# select * from doctor;
 name | on_call
-----+-----
   Amit | t
  Rahul | t
  Neha | f
(3 rows)
lovabledb=#

```

- ⌘ Here, I want to change just one field.
- ⌘ In case of REPEATABLE READ, I would have ended up changing 2 columns to be false (last example of REPEATABLE READ) but I didn't want that. I wanted to change only one row.

```

lovabledb=# begin; 1
BEGIN
lovabledb=# show transaction isolation level; 2
transaction_isolation
-----
read committed
(1 row)

lovabledb=# set transaction isolation level serializable; 3
SET
lovabledb=# show transaction isolation level; 4
transaction_isolation
-----
serializable
(1 row)

lovabledb=# select * from accounts; 5
 name | balance
-----+-----
Neha  | 1200.75
Amit   | 365.00
Rahul  | 385.00
(3 rows)

lovabledb=# update accounts set balance=balance+15 where name='Amit'; 6
UPDATE 1
lovabledb=# select * from accounts; 8
 name | balance
-----+-----
Neha  | 1200.75
Rahul | 385.00
Amit   | 380.00
(3 rows)

lovabledb=# commit; 10
COMMIT
lovabledb=# select * from accounts; 12
 name | balance
-----+-----
Neha  | 1200.75
Rahul | 385.00
Amit   | 380.00
(3 rows)

lovabledb=#

```

```

lovabledb=# begin; 1
BEGIN
lovabledb=# show transaction isolation level; 2
transaction_isolation
-----
read committed
(1 row)

lovabledb=# set transaction isolation level serializable; 3
SET
lovabledb=# show transaction isolation level; 4
transaction_isolation
-----
serializable
(1 row)

lovabledb=# select * from accounts; 5
 name | balance
-----+-----
Neha  | 1200.75
Amit   | 365.00
Rahul  | 385.00
(3 rows)

lovabledb=# update accounts set balance=balance+20 where name='Rahul'; 7
UPDATE 1
lovabledb=# select * from accounts; 9
 name | balance
-----+-----
Neha  | 1200.75
Amit   | 365.00
Rahul  | 405.00
(3 rows)

lovabledb=# commit; 11
ERROR: could not serialize access due to read/write dependencies among transactions
DETAIL: Reason code: Canceled on identification as a pivot, during commit attempt.
HINT: The transaction might succeed if retried.
lovabledb=#
lovabledb=# select * from accounts; 13
 name | balance
-----+-----
Neha  | 1200.75
Rahul | 385.00
Amit   | 380.00
(3 rows)

```

~ It could have updated both if it was REPEATABLE READ.

➤ Consider the below example

```

lovabledb=# select * from doctor;
 name | on_call
-----+-----
Amit   | t
Rahul  | t
Neha   | f
(3 rows)

```

(initial state)

```

lovabledb=# begin; 1
BEGIN
lovabledb=# show transaction isolation level; 2
transaction_isolation
-----
read committed
(1 row)

lovabledb=# set transaction isolation level serializable; 3
SET
lovabledb=# show transaction isolation level; 4
transaction_isolation
-----
serializable
(1 row)

lovabledb=# select * from doctor where name='Amit'; 5
 name | on_call
-----+-----
Amit   | t
(1 row)

lovabledb=# update doctor set on_call=false where name='Amit'; 7
UPDATE 1
lovabledb=# commit; 10
ERROR: could not serialize access due to read/write dependencies among transactions
DETAIL: Reason code: Canceled on identification as a pivot, during commit attempt.
HINT: The transaction might succeed if retried.
lovabledb=#

```

```

lovabledb=# begin; 1
BEGIN
lovabledb=# show transaction isolation level; 2
transaction_isolation
-----
read committed
(1 row)

lovabledb=# set transaction isolation level serializable; 3
SET
lovabledb=# show transaction isolation level; 4
transaction_isolation
-----
serializable
(1 row)

lovabledb=# select * from doctor where name='Rahul'; 6
 name | on_call
-----+-----
Rahul | t
(1 row)

lovabledb=# update doctor set on_call=false where name='Rahul'; 8
UPDATE 1
lovabledb=# commit; 9
COMMIT
lovabledb=#

```

~ The one who **commits first** becomes the winner.

➤ But its not the rule of Serializable; in PostgreSQL it seems like only one updation in a table is possible in case of concurrent transactions, but it should be like till the transactions are not conflicting each other, they should commit without any error.

- ⌘ If two concurrent transactions start from the same snapshot and both perform writes, PostgreSQL may abort one if their effects cannot be ordered into a valid serial schedule.

- If you use SERIALIZABLE, then it'll reduce the concurrency. But more secure.

Table 13.1. Transaction Isolation Levels

Isolation Level	Dirty Read	Nonrepeatable Read	Phantom Read	Serialization Anomaly
Read uncommitted	Allowed, but not in PG	Possible	Possible	Possible
Read committed	Not possible	Possible	Possible	Possible
Repeatable read	Not possible	Not possible	Allowed, but not in PG	Possible
Serializable	Not possible	Not possible	Not possible	Not possible

⌘

⌘ Top to bottom:

- ⌘ **Decreasing error due to more strict.**
- ⌘ **Decreasing concurrency.**

- F

===== @Transactional in Spring Boot =====

- **@Transactional** is a AOP related concept in Spring Boot.
 - ⌘ Beginning of transaction: before the method starts executing, spring opens a DB connection and begins a transaction.
 - ⌘ Committing: if the method completes successfully without throwing any exception, spring will commit the transaction persisting all changes to the DB.
 - ⌘ Rolling back: if the method throws an unchecked exception (RuntimeException or Error), the transaction is rolled back.
- There is a field **propagation** in **@Transactional** annotation which is used to replicate transactional context

Propagation	Calling method (outer)	Called method (inner)
REQUIRED	No	T2
REQUIRED	T1	T1
REQUIRES_NEW	No	T2
REQUIRES_NEW	T1	T2
MANDATORY	No	Exception
MANDATORY	T1	T1
NOT_SUPPORTED	No	No
NOT_SUPPORTED	T1	No
SUPPORTS	No	No
SUPPORTS	T1	T1
NEVER	No	No
NEVER	T1	Exception
NESTED	No	T2
NESTED	T1	T2

- ⌘ The propagation is given in the **@Transactional** annotation which is being annotated with called method (inner)
- ⌘ If the called method has **REQUIRED**, then it'll use its own transactional context if there no existing transaction (i.e. in the calling method); if calling method is a transaction, then called method will not create another.
- Inside **@Transactional** you can set the *isolation level* and *propagation*.

```
public @interface Transactional {
    @AliasFor("transactionManager")
    String value() default "";

    @AliasFor("value")
    String transactionManager() default "";

    String[] label() default {};

    Propagation propagation() default Propagation.REQUIRED;

    Isolation isolation() default Isolation.DEFAULT;
```

```
public enum Propagation {  
    REQUIRED(0),  
    SUPPORTS(1),  
    MANDATORY(2),  
    REQUIRES_NEW(3),  
    NOT_SUPPORTED(4),  
    NEVER(5),  
    NESTED(6);  
}
```

^

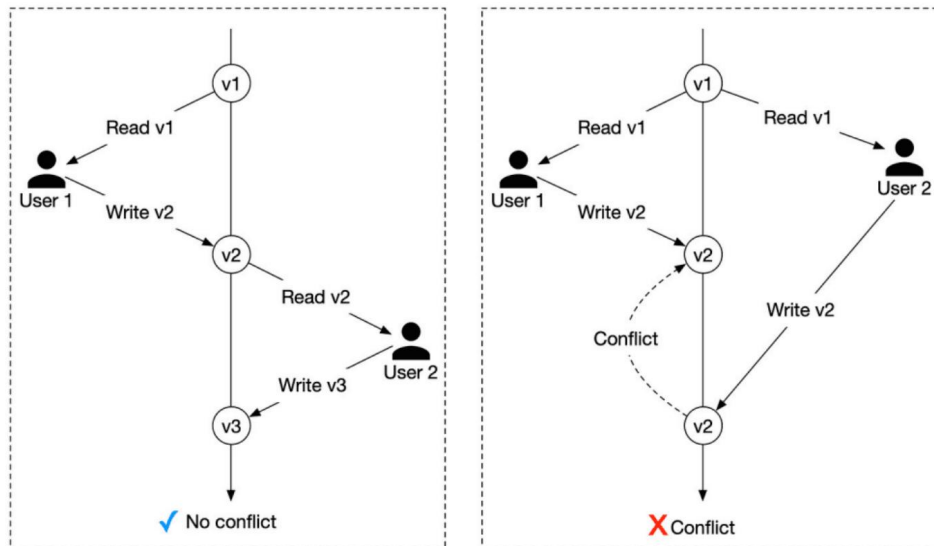
```
public enum Isolation {  
    DEFAULT(-1),  
    READ_UNCOMMITTED(1),  
    READ_COMMITTED(2),  
    REPEATABLE_READ(4),  
    SERIALIZABLE(8);  
}
```

^

➤ F

===== Transactional Lock in Spring Data JPA =====

- Don't get confused about **isolation level** and **lock**. Both are inter-linked.
 - ↪ Isolation Level is the **rule** (analogy: law)
 - ↪ Lock is the **implementation** of those rule (analogy: police)
 - ↪ BUT, Lock is not the *only implementation* of isolation level. Its one of those many implementations of isolation level. (ex: postgres uses MVCC for isolation level implementation)
 - ↪ If you want to specify the lock explicitly without depending upon the isolation levels (because different databases implement isolation level differently) then you can do it.
 - ↪ Isolation levels define the concurrency guarantees (rules). Locks are one of the mechanisms databases use to enforce those guarantees, along with techniques like MVCC. Explicit locking allows an application to impose stricter, operation-specific concurrency control rather than relying solely on the default behavior of the isolation level.
- Spring itself doesn't create locks; rather it delegates the locking to the DB via JPA/Hibernate.
- Transactional lock:
 - ↪ When a transaction access a row, the DB decides who can read/write it until the transaction finishes.
 - ↪ 2 types of lock in Spring Data JPA
 - ↪ Optimistic Locking
 - ↪ Pessimistic Locking
- **Optimistic Lock**
 - ↪ It keeps a version column in the table.
 - ↪ If someone update that row, then it's version will be increased.
 - ↪ If 2 transactions read the same row(let version = 0),
 - ↪ one update the data and commit. (version = 1)
 - ↪ Now if another transaction try to update, it'll fail because it will make the version 1 (because it was 0 when it read). but to update, version needs to be 2.



- ♣ To implement Optimistic Lock in Spring Data JPA, use `@Version` to a column (column name doesn't matter)

```
@Version
private Long version;
```

- ♣ You'll get an extra column "version"

123 balance	123 employee_id	123 id	123 version
94	1	1	94

- ♣ Command to make a request concurrently multiple times

```
seq 100 | xargs -P 10 -I {} curl --location --request PUT 'http://127.0.0.1:8080/employees/incrementBalance/1'
```

- ♣ Now you'll get error like

```
ERROR 51612 --- [cachingApp] [nio-8080-exec-7] c.c.c.advices.GlobalExceptionHandler : Row was updated or deleted by another transaction
```

➤ Pessimistic Lock

- ♣ In Optimistic lock, other can see the row and update if the versioning is proper.
- ♣ But in Pessimistic lock you can control the write and read of the row by the other transactions if a transaction has locked already.
- ♣ Pessimistic locking is a concurrency control mechanism where a transaction locks a database row immediately to prevent other transactions from **modifying** it until the lock is released.
You can also prevent **reading** the row by other transactions.
- ♣ 2 types of Pessimistic lock:
 - ♣ PESSIMISTIC READ
 - * Other transaction can read but cannot modify the row.
 - ♣ PESSIMISTIC WRITE

* Other transactions can neither read nor modify the row.

⌘ It is being written in the repository methods

```
@Override 3 usages new *
@Lock(LockModeType.PESSIMISTIC_WRITE)
Optional<SalaryAccount> findById(Long id);
```

(version is also there, I have not removed yet)

```
select
  id
from
  salary_account
where
  id=?
  and version=? for update
```

⌘ (now for all findById, it'll lock the row)

⌘ Now, every transaction will wait till the lock is set free, then they will lock and modify.

Hence, if you trigger 100 concurrent request now, still they will modify the DB sequentially, so now if you increment that value 100 times then it'll be exact 100 now.

NOTE: the method that is modifying the entity should be annotated with

@Transactional

```
@Transactional
public SalaryAccountResponse incrementBalance(Long accountId) {
    SalaryAccount salaryAccount = salaryAccountRepository.findById(id: accountId)
        .orElseThrow(exceptionSupplier: () -> new RuntimeException("Account not found"));

    BigDecimal prevBalance = salaryAccount.getBalance();
    BigDecimal newBalance = prevBalance.add(BigDecimal.valueOf(val: 1L));

    salaryAccount.setBalance(newBalance);

    return modelMapper.map(source: salaryAccountRepository.save(entity: salaryAccount), destina
}
```

And, if the fetch type is eager (in case of relation like one-to-one, many-to-one etc) then also it'll not make the value 100

```
public class SalaryAccount {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    Long id;

    private BigDecimal balance;

    @OneToOne(fetch = FetchType.LAZY)
    private Employee employee;

    @Version
    private Long version;
}
```

⌘ Now the result will be 100

```
$ seq 100 | xargs -P 10 -I {} curl --location --request PUT 'http://127.0.0.1:8080/employees/incrementBalance/1'
{"id":1,"balance":2.00}{id":1,"balance":1.00}{id":1,"balance":3.00}{id":1,"balance":4.00}{id":1,"balance":5.00}{id":1,"balance":11.00}{id":1,"balance":12.00}{id":1,"balance":13.00}{id":1,"balance":14.00}{id":1,"balance":20.00}{id":1,"balance":21.00}{id":1,"balance":22.00}{id":1,"balance":23.00}{id":1,"balance":24.00}{id":1,"balance":30.00}{id":1,"balance":31.00}{id":1,"balance":32.00}{id":1,"balance":33.00}{id":1,"balance":39.00}{id":1,"balance":40.00}{id":1,"balance":41.00}{id":1,"balance":42.00}{id":1,"balance":43.00}{id":1,"balance":49.00}{id":1,"balance":50.00}{id":1,"balance":51.00}{id":1,"balance":52.00}{id":1,"balance":53.00}{id":1,"balance":59.00}{id":1,"balance":60.00}{id":1,"balance":61.00}{id":1,"balance":62.00}{id":1,"balance":68.00}{id":1,"balance":69.00}{id":1,"balance":70.00}{id":1,"balance":71.00}{id":1,"balance":72.00}{id":1,"balance":78.00}{id":1,"balance":79.00}{id":1,"balance":80.00}{id":1,"balance":81.00}{id":1,"balance":87.00}{id":1,"balance":88.00}{id":1,"balance":89.00}{id":1,"balance":90.00}{id":1,"balance":91.00}{id":1,"balance":97.00}{id":1,"balance":98.00}{id":1,"balance":99.00}{id":1,"balance":100.00}
```

⌘ **stale data** means the *version* is already updated and current updation is giving conflict. (in Optimistic Lock)

➤ There are 3 types of read queries in PostgreSQL

- ♣ **FOR UPDATE, FOR SHARE,** and plain select query.

♣ **SELECT * FROM account WHERE id=1;**

- ♣ It's the plain read query.
- ♣ It doesn't apply any lock and it can be executed at any condition.
- ♣ It'll just get the data from the latest snapshot.
- ♣ NO LOCK

♣ **SELECT * FROM account WHERE id=1 FOR UPDATE;**

- ♣ It applies row lock.
- ♣ It blocks the following
 - UPDATE*
 - DELETE*
 - SELECT FOR UPDATE*
 - SELECT FOR SHARE*

♣ **SELECT * FROM account WHERE id=1 FOR SHARE;**

- ♣ It applies row lock.
- ♣ It blocks
 - UPDATE*
 - DELETE*
- ♣ It allows
 - FOR SHARE*
 - SELECT* (plain select)

===== UPDATE, FOR UPDATE, FOR SHARE =====

➤ **UPDATE**

- ♣ It directly modifies the row in the table without reading anything.

♣ `UPDATE account SET balance = 500 WHERE id = 1;`

➤ **SELECT ... FOR UPDATE**

- ♣ It just applies a lock, it is like “*I want to read the row first, then I’ll safely modify*”
- ♣ It applies lock so that no one can modify this.
- ♣ Also no one can apply *SHARE* lock if FOR UPDATE lock is already applied on a row.

♣ `SELECT * FROM account WHERE id = 1 FOR UPDATE;`

➤ **SELECT ... FOR SHARE**

- ♣ It applies the lock so that no one can modify the row until it commits.
- ♣ It doesn’t modify the row, it just ensures that till I am querying this row no one else should modify this.
- ♣ Its like “I will use this row now but not modify it, till then no one should modify this (UPDATE, FOR UPDATE cannot be applied), but they can apply FOR SHARE lock (understandable, because even if someone applies FOR SHARE lock they cannot modify it which preserves the intent)

♣ `SELECT * FROM account WHERE id=1 FOR SHARE;`

➤ In summary:

- ♣ **UPDATE** directly changes the row and takes an exclusive lock automatically.
- ♣ **SELECT ... FOR UPDATE** does not modify the row, but takes an exclusive lock so you can safely read it first and then update it. No other transaction can modify or acquire a *share/update* lock on that row until you commit.
- ♣ **SELECT ... FOR SHARE** also does not modify the row, but takes a shared lock to ensure no one modifies it while you’re reading it. Other transactions can also take a share lock, but they still cannot update it until all share locks are released.

➤ In short:

- ♣ **UPDATE** : directly update the row, holds the *lock* till transaction commits/rolls-back.

```
BEGIN;
UPDATE account SET balance = 500 WHERE id = 1;
-- Row is locked here
-- Lock stays until:
COMMIT; -- or ROLLBACK
```

- ⌘ **FOR UPDATE** : applies lock, holds the *lock* till transaction commits/rolls-back.

```
BEGIN;  
SELECT * FROM account WHERE id = 1 FOR UPDATE;  
-- Row is locked (exclusive lock)  
-- Lock stays until:  
COMMIT; -- or ROLLBACK
```

- ⌘ No one allowed to applies lock on that row (FOR SHARE, UPDATE, FOR UPDATE)

- ⌘ **FOR SHARE** : applies lock, holds the *lock* till transaction commits/rolls-back.

```
BEGIN;  
SELECT * FROM account WHERE id = 1 FOR SHARE;  
-- Row is locked (shared lock)  
-- Lock stays until:  
COMMIT; -- or ROLLBACK
```

- ⌘ Other can applies lock (FOR SHARE) but cannot FOR UPDATE, UPDATE.

➤ In spring, **PESSIMISTIC_READ** means **FOR SHARE**, **PESSIMISTIC_WRITE** means **FOR UPDATE** lock.

```
Exclusive locks (FOR UPDATE) → high deadlock risk  
Shared locks (FOR SHARE) → lower risk, but possible if upgraded
```



➤ How actually **PESSIMISTIC READ** and **WRITE** works?

⌘ In case of **PESSIMISTIC_WRITE**, it makes exclusive row level lock

⌘ T1 does below

```
BEGIN;  
SELECT * FROM account WHERE id=1 FOR UPDATE;
```

(postgres query

for locking)

⌘ Now if **T2** tries to update something, it'll have to wait till T2 commits or rollback

```
UPDATE account SET balance=900 WHERE id=1;
```

(T2, it has to

wait)

⌘ If **T2** tries to even read, then also it has to wait

```
SELECT * FROM account WHERE id=1 FOR UPDATE;
```

➤ F

➤ Some experiments:

⌘ ---- I removed @Version and kept PESSIMISTIC_READ ----

```
@Override 3 usages new *
@Lock(LockModeType.PESSIMISTIC_READ)
Optional<SalaryAccount> findById(Long id);
```

(repository)

```
// @Version
private Long version;
```

(entity)

```
@Override 1 usage new *
// @Transactional
public SalaryAccountResponse incrementBalance(Long accountId) {
```

(removed the @Transactional annotation as well)

Now if I trigger 100 requests concurrently, **Dead-Lock** will be occurred.

```
2026-03-03T14:04:43.718+05:30 ERROR 43640 --- [cachingApp] [io-8080-exec-10] o.h.engine.jdbc.spi.SqlExceptionHelper : ERROR: deadlock detected
Detail: Process 516 waits for ShareLock on transaction 2974; blocked by process 517.
Process 517 waits for ShareLock on transaction 2973; blocked by process 516.
```

Even with keeping @Transactional annotation, this deadlock will occur.

Here, we are using **FOR SHARE** lock (PESSIMISTIC_READ), which means other transactions can also apply **FOR SHARE** lock, (*but we are updating the row in our code which shouldn't be done for FOR SHARE lock*), so when we are executing update query, it'll not work because other transactions might also have applied **FOR SHARE** lock.

So, if you use PESSIMISTIC_WRITE (FOR UPDATE) lock, then it'll not permit other transactions to apply lock, they need to wait until the current transaction commits or rolls back. So in this case, **deadlock will not occur**.

➤ F

➤ F

➤ F

➤ F

➤